

INSTITUTO FEDERAL DE EDUCACAO, CIENCIA E TECNOLOGIA DO TOCANTINS
CAMPUS PARAISO DO TOCANTINS
CURSO DE BACHARELADO EM SISTEMAS DE INFORMACAO

DOCUMENTO DE ESPECIFICACAO:
T.O.D.A.N (TRUTH OR DARE ASYNC NETWORK)

Disciplina: Linguagens e Tecnicas de Programacao IV

Professor: Marcos Raimundo Mendes Ramos

Equipe de Desenvolvimento:

- Pedro Roberto Ribeiro Bandeira Labre

Paraiso do Tocantins - 2026

1. HISTORICO DE ALTERACOES

Data	Versao	Descricao da Alteracao	Autor
24/03/2026	1.0	Inicio da fase de prototipacao visual (light mode) e definicao inicial do escopo do aplicativo.	Pedro Roberto Ribeiro Bandeira Labre
25/03/2026	1.1	Evolucao dos prototipos light mode e organizacao dos fluxos no README para documentacao visual.	Pedro Roberto Ribeiro Bandeira Labre
26/03/2026	1.2	Conclusao da rodada de prototipos dark mode (busca, perfil, clubes, grupos, respostas e configuracoes).	Pedro Roberto Ribeiro Bandeira Labre
28/03/2026	1.3	Inicializacao tecnica do backend (Express, Prisma, PostgreSQL) e implementacao da autenticacao com testes automatizados.	Pedro Roberto Ribeiro Bandeira Labre
28/03/2026	1.4	Inicializacao da base do app mobile com tela de login e implementacao da tela de cadastro integrada ao fluxo de autenticacao.	Pedro Roberto Ribeiro Bandeira Labre
29/03/2026	1.5	Integracao da autenticacao mobile com backend e consolidacao dos testes de frontend para login e cadastro.	Pedro Roberto Ribeiro Bandeira Labre
30/03/2026	1.6	Implementacao da tela de feed mobile e configuracao de integracao da API por variaveis de ambiente.	Pedro Roberto Ribeiro Bandeira Labre
01/04/2026	1.7	Implementacao do backend do feed com testes e isolamento do banco de testes para execucoes automatizadas.	Pedro Roberto Ribeiro Bandeira Labre
01/04/2026	1.8	Entrega de criacao de truths e dares no backend, com cobertura de testes e integracao com feed.	Pedro Roberto Ribeiro Bandeira Labre
01/04/2026	1.9	Implementacao da tela create-challenge no mobile com consumo real de usuarios, truths e dares.	Pedro Roberto Ribeiro Bandeira Labre
04/04/2026	1.10	Aprimoramentos do feed: persistencia de usuario-alvo, configuracoes de dare e regras de permissao para delete.	Pedro Roberto Ribeiro Bandeira Labre
05/04/2026	1.11	Integracao de status real de dares e likes persistidos no feed, com melhorias de comportamento de interface.	Pedro Roberto Ribeiro Bandeira Labre
06/04/2026	1.12	Implementacao da tela de comentarios do feed no mobile, com estrutura de fluxo pronta para evolucao backend.	Pedro Roberto Ribeiro Bandeira

Data	Versao	Descricao da Alteracao	Autor
			Labre
07/04/2026	1.13	Implementacao da base de perfil, configuracoes e notificacoes no mobile com tema global compartilhado.	Pedro Roberto Ribeiro Bandeira Labre
08/04/2026	1.14	Consolidacao das telas de busca, clubes e criacao de grupo; integracao de perfil real (username/bio) e logout funcional.	Pedro Roberto Ribeiro Bandeira Labre
08/04/2026	1.15	Atualizacao da documentacao com analise tecnica detalhada de backend e mobile, incluindo escopo implementado e integracoes pendentes.	Pedro Roberto Ribeiro Bandeira Labre
08/04/2026	1.16	Ajuste do build de producao do backend para viabilizar deploy online no Render com banco Supabase (commit 0e16e75).	Pedro Roberto Ribeiro Bandeira Labre
08/04/2026	1.17	Correcao dos imports do Prisma Client no artefato compilado para estabilidade do backend online no Render (commit 17cbd76).	Pedro Roberto Ribeiro Bandeira Labre
08/04/2026	1.18	Ajuste do prisma.config para migrations com DIRECT_URL do Supabase, garantindo fluxo de deploy (commit a32427f).	Pedro Roberto Ribeiro Bandeira Labre
08/04/2026	1.19	Correcao do entypoint de start em producao para manter a API online no Render (commit 4a6ffb1).	Pedro Roberto Ribeiro Bandeira Labre
08/04/2026	1.20	Consolidacao da publicacao do backend em nuvem: aplicacao no Render e banco PostgreSQL gerenciado no Supabase.	Pedro Roberto Ribeiro Bandeira Labre
08/04/2026	1.20.1	Terceira entrega correspondente a etapa final da primeira versao do aplicativo.	Pedro Roberto Ribeiro Bandeira Labre
09/04/2026	1.21	[ULTIMA ENTREGA] Implementacao da interface completa do fluxo de recuperacao de senha no mobile.	Pedro Roberto Ribeiro Bandeira Labre
10/04/2026	1.22	[ULTIMA ENTREGA] Ajuste de SSL do Prisma runtime para conexao com Supabase.	Pedro Roberto Ribeiro Bandeira Labre
04/05/2026	1.23	[ULTIMA ENTREGA] Implementacao do backend de proofs e base de envio de midias, com estrutura inicial de proof detail no mobile.	Pedro Roberto Ribeiro Bandeira Labre
05/05/2026	1.24	[ULTIMA ENTREGA] Implementacao da visualizacao local de proofs no mobile.	Pedro Roberto Ribeiro Bandeira Labre

Data	Versao	Descricao da Alteracao	Autor
06/05/2026	1.25	[ULTIMA ENTREGA] Implementacao de comentarios reais em truths.	Pedro Roberto Ribeiro Bandeira Labre
08/05/2026	1.26	[ULTIMA ENTREGA] Aprimoramento de comentarios e denuncias de truths, com acoes e cobertura de testes.	Pedro Roberto Ribeiro Bandeira Labre
09/05/2026	1.27	[ULTIMA ENTREGA] Implementacao do backend de clubes (schema, CRUD, membros, convites e solicitacoes).	Pedro Roberto Ribeiro Bandeira Labre
10/05/2026	1.28	[ULTIMA ENTREGA] Administracao de membros e auditoria de clubes, com publicacao de prompts.	Pedro Roberto Ribeiro Bandeira Labre
11/05/2026	1.29	[ULTIMA ENTREGA] Prompts de clubes (detalhe, edicao, moderacao, respostas, comentarios, likes) e feed interno.	Pedro Roberto Ribeiro Bandeira Labre
12/05/2026	1.30	[ULTIMA ENTREGA] Implementacao de feed agregado de clubes.	Pedro Roberto Ribeiro Bandeira Labre
13/05/2026	1.31	Reformulacao dos requisitos que ja foram implementados e dos requisitos futuros.	Pedro Roberto Ribeiro Bandeira Labre

2. DEFINICAO DO PROJETO

2.1 Tema

Rede social assincrona baseada em Verdade ou Desafio, com interacoes por prompts de verdade/desafio, feed social e dinamicas em grupos (clubs).

2.2 Objetivo do Aplicativo

O aplicativo busca oferecer uma experiencia social de entretenimento em que usuarios criam e respondem desafios de forma assincrona, sem depender de todos estarem conectados ao mesmo tempo. O foco e estimular engajamento por meio de perfis, feed, curtidas e participacao em grupos.

3. REQUISITOS DO SISTEMA

3.1 Requisitos Funcionais (RF)

Descrevem as acoes que o sistema deve ser capaz de executar.

- RF001: Acessar login: o app mobile deve exibir a tela de login.
- RF002: Acessar cadastro: o app mobile deve exibir a tela de cadastro.
- RF003: Cadastrar usuario: o sistema deve permitir cadastro de usuario por email e senha.
- RF004: Autenticar usuario: o sistema deve permitir autenticacao de usuario por email e senha e emitir token JWT no login autenticado.
- RF005: Armazenar token: o app mobile deve persistir o token de autenticacao localmente.

- RF006: Autorizar requisicoes: o app mobile deve enviar o token nas requisicoes protegidas.
- RF007: Consultar feed: o app mobile deve exibir a tela de feed com itens recentes.
- RF008: Consultar comentarios: o app mobile deve exibir a tela de comentarios do feed.
- RF009: Curtir truth: o sistema deve permitir curtir e descurtir truths.
- RF010: Curtir dare: o sistema deve permitir curtir e descurtir dares.
- RF011: Curtir club: o sistema deve permitir curtir e descurtir clubs.
- RF012: Excluir verdade: o sistema deve permitir excluir uma verdade quando houver permissao.
- RF013: Excluir desafio: o sistema deve permitir excluir um desafio quando houver permissao.
- RF014: Iniciar criacao de desafio: o app mobile deve exibir a tela de criacao de desafio.
- RF015: Listar usuarios: o sistema deve listar usuarios para interacao no fluxo de criacao de desafio.
- RF016: Criar verdade: o sistema deve permitir criar prompts do tipo verdade para outro usuario.
- RF017: Criar desafio: o sistema deve permitir criar prompts do tipo desafio para outro usuario.
- RF018: Acessar perfil: o app mobile deve exibir a tela de perfil.
- RF019: Consultar perfil: o app mobile deve carregar dados do proprio perfil pela API.
- RF020: Atualizar perfil: o app mobile deve atualizar dados do proprio perfil pela API.
- RF021: Acessar configuracoes: o app mobile deve exibir a tela de configuracoes.
- RF022: Encerrar sessao: o app mobile deve realizar logout removendo token local.
- RF023: Redirecionar login: o app mobile deve redirecionar para login apos logout.
- RF024: Acessar busca: o app mobile deve exibir a tela de busca.
- RF025: Acessar clubes: o app mobile deve exibir a tela de clubes.
- RF026: Acessar criacao de grupo: o app mobile deve exibir a tela de criacao de grupo.

3.2 Requisitos Nao Funcionais (RNF)

Descrevem caracteristicas de qualidade, desempenho e restricoes.

Comunicacao: A comunicacao entre o app e o servidor (API) deve ocorrer via protocolo web seguro e padronizado (HTTP). Os dados devem ser trocados em formato estruturado e consistente (JSON).

Seguranca: Requisicoes protegidas devem exigir autenticao por token (JWT). Senhas devem ser armazenadas com hash seguro (bcrypt) para reduzir riscos de vazamento.

Backend: O backend deve ser implementado em TypeScript para padronizar tipagem e manutencao. O backend deve compilar via TypeScript compiler (tsc) para gerar o build de producao.

Testes automatizados: O backend deve possuir testes automatizados com Jest para validar regras e rotas. O mobile deve possuir testes automatizados com Jest para validar telas e fluxos.

Banco de dados: A persistencia deve utilizar PostgreSQL como banco relacional. O acesso ao banco deve utilizar Prisma ORM para mapear modelos e consultas.

Versionamento: O codigo-fonte deve ser versionado com Git para historico e rastreio. O repositorio deve ser mantido no GitHub para colaboracao.

Navegacao mobile: O mobile deve utilizar Expo Router para navegacao baseada em rotas. O mobile deve manter stack centralizada de rotas para controle do fluxo.

Temas do mobile: O mobile deve suportar modo automatico (system) seguindo o sistema, modo claro (light) e modo escuro (dark) para acessibilidade.

Arquitetura mobile: O token de autenticao no mobile deve ser persistido com AsyncStorage para manter a sessao. O consumo de API no mobile deve ser centralizado em camada de servico para consistencia. O consumo de API no mobile deve aplicar tratamento padrao de erro para mensagens

previsíveis. A arquitetura mobile deve separar componentes reutilizáveis, hooks de estado e fluxo, constantes de tema e UI e tipos compartilhados.

Deploy do backend: O build de produção do backend deve executar prisma generate antes da compilação TypeScript para gerar o cliente. O backend em produção deve iniciar pelo endpoint compilado dist/src/server.js para garantir o runtime correto. As migrations do backend devem usar conexão direta via variável DIRECT_URL. O runtime do backend deve conectar no banco pela variável DATABASE_URL. O servidor backend deve fazer bind em 0.0.0.0 com porta via variável PORT para atender o ambiente de deploy. O backend deve estar publicado em ambiente de produção no Render. O banco de produção deve utilizar PostgreSQL gerenciado no Supabase. O deploy de produção deve aplicar migrations com prisma migrate deploy antes do start da API.

Variáveis de ambiente: O backend deve exigir as variáveis DATABASE_URL, DIRECT_URL e JWT_SECRET no ambiente de produção. O mobile deve exigir a variável EXPO_PUBLIC_API_URL para consumo da API publicada.

3.3 Requisitos Futuros

Requisitos funcionais planejados a partir do estado atual do sistema, cobrindo clubes (CRUD, membros, convites e auditoria), feed de clubes, prompts, comentários, provas, uploads e funcionalidades mobile (recuperação de senha, busca, notificações e configurações).

- RFF01: Gerenciar clube: o sistema deve permitir criar, atualizar, consultar detalhes, arquivar e restaurar clubes.
- RFF02: Descobrir clubes: o sistema deve permitir listar meus clubes, descobrir clubes disponíveis e buscar clubes por critério.
- RFF03: Gerenciar membros: o sistema deve permitir listar membros com filtros e busca, alterar papel, remover membro e transferir propriedade.
- RFF04: Gerenciar entradas: o sistema deve permitir criar convites, listar convites, aceitar/recusar convites, entrar em clubes públicos, solicitar entrada em clubes privados, aprovar/rejeitar solicitações e sair do clube.
- RFF05: Notificações de clube: o sistema deve permitir silenciar e desilenciar notificações do clube.
- RFF06: Auditar ações: o sistema deve registrar e permitir consultar auditoria de ações de membros.
- RFF07: Feed de clubes: o sistema deve disponibilizar feed agregado e feed do clube.
- RFF08: Gerenciar prompts: o sistema deve permitir criar/publicar, editar, detalhar, moderar e definir campos e permissões de prompts de clube.
- RFF09: Interagir com prompts: o sistema deve permitir curtir prompt, responder prompt, listar respostas, curtir resposta e comentar prompt de clube.
- RFF10: Comentários de truths: o sistema deve permitir listar, criar, curtir e responder comentários de truths.
- RFF11: Enviar prova: o sistema deve permitir enviar prova de desafio com mídia e texto opcional.
- RFF12: Gerar upload: o sistema deve permitir gerar URL assinada para upload de mídia.
- RFF13: Visualizar prova: o app mobile deve disponibilizar tela de detalhes da prova do desafio.
- RFF14: Solicitar código: o app mobile deve permitir solicitar código de recuperação de senha.
- RFF15: Reenviar código: o app mobile deve permitir reenviar código de recuperação de senha.
- RFF16: Validar código: o app mobile deve permitir validar código de recuperação de senha.
- RFF17: Redefinir senha: o app mobile deve permitir redefinir senha.
- RFF18: Buscar usuários e clubes: o app mobile deve permitir buscar usuários e clubes por termo e visualizar resultados.

- RFF19: Exibir sugestões: o app mobile deve exibir usuários recomendados e clubes em alta no estado inicial da busca.
- RFF20: Gerenciar recentes: o app mobile deve permitir salvar, remover e limpar buscas recentes.
- RFF21: Abrir resultado: o app mobile deve permitir abrir perfil público de usuário ou detalhe de clube a partir da busca.
- RFF22: Listar notificações: o app mobile deve permitir listar notificações do usuário.
- RFF23: Marcar notificação: o app mobile deve permitir marcar notificação como lida.
- RFF24: Marcar todas: o app mobile deve permitir marcar todas as notificações como lidas.
- RFF25: Exibir não lidas: o app mobile deve exibir a contagem de notificações não lidas.
- RFF26: Abrir notificação: o app mobile deve permitir navegar para o conteúdo relacionado a notificação.
- RFF27: Carregar dados: o app mobile deve carregar dados reais do usuário na tela de configurações.
- RFF28: Alterar privacidade: o app mobile deve permitir alternar conta privada/pública.
- RFF29: Alterar email: o app mobile deve permitir alterar o email da conta.
- RFF30: Alterar senha: o app mobile deve permitir alterar a senha da conta.
- RFF31: Persistir tema: o app mobile deve persistir a preferência de tema entre sessões.
- RFF32: Solicitar suporte: o app mobile deve permitir enviar denúncia de abuso ou contato com suporte.
- RFF33: Excluir conta: o app mobile deve permitir excluir a própria conta.

3.4 Requisitos Futuros Não Funcionais

Requisitos de qualidade e restrições planejados para evoluções futuras do sistema.

Desempenho: Buscas, notificações e feeds de clubes devem responder com latência consistente, usando paginação (cursor/limit) e payloads enxutos.

Escalabilidade: O backend deve suportar crescimento de clubes, prompts e notificações com índices adequados e limites por requisição.

Segurança: O fluxo de recuperação de senha deve usar tokens com expiração, hash seguro (bcrypt/SHA-256), escopo de reset e limitação de tentativas.

Privacidade: A busca deve respeitar contas privadas e visibilidade de clubes, evitando exposição de dados sensíveis em resultados ou notificações.

Confiabilidade: A criação de notificações deve ser idempotente (deduplicação) para evitar repetição por eventos iguais.

Disponibilidade: Falhas em serviços externos (ex.: envio de e-mail) não devem interromper o fluxo do usuário; o sistema deve degradar com retorno genérico e log interno.

Consistência: Operações de clubes (membros, convites, prompts) devem manter contadores e audit log consistentes, preferindo transações.

Retenção: Notificações devem seguir política de retenção (ex.: 90 dias) e permitir limpeza segura sem perda de integridade.

Experiência: A busca deve aplicar debounce e cancelamento de requisições para evitar chamadas excessivas e travamentos na UI.

Persistência local: Preferências do app (tema, estado de filtros) devem ser armazenadas em AsyncStorage e restauradas na inicialização.

Observabilidade: O sistema deve registrar logs estruturados e metricas basicas para recuperacao de senha, notificacoes e busca, sem registrar dados sensiveis.

Acessibilidade: As telas futuras devem manter contraste adequado, rotulos de acessibilidade e alvos de toque coerentes.

3.5 Modelagem

Modelagem de alto nivel das funcionalidades e telas do aplicativo.

Modulo	Entidades/Recursos	Telas/Fluxos	Status mobile atual
Autenticacao	User, JWT	login, signup-screen	Integrado com API (/auth/signup, /auth/login) e persistencia de token.
Feed	Truth, Dare	feed, feed-comments, create-challenge	Feed e create-challenge integrados; comentarios com estrutura pronta para backend.
Perfis	User	profile, settings	Perfil integrado em /users/me (GET/PUT) e logout funcional em settings.
Interacoes	Like (truth/dare/club)	feed, cards de interacao	Like/deslike integrado com endpoints de truth, dare e club.
Clubes/Grupos	Club, ClubMember, ClubPrompt	clubs, create-group, search	Fluxos e UI prontos; integracao backend pendente para dados reais.

4. REQUISITOS TECNICOS DO PROJETO

Definicao da arquitetura e ferramentas utilizadas.

Linguagem	TypeScript (backend), TypeScript/TSX (mobile)
Framework / Plataforma	Express 5 (backend), React Native + Expo + Expo Router (mobile)
Deploy Backend (estado atual)	Build de producao: prisma generate && tsc -p tsconfig.build.json. Start de producao: node dist/src/server.js. Prisma migrations: prisma.config.ts com datasource.url em env('DIRECT_URL'). Runtime Prisma: adapter PostgreSQL usando process.env.DATABASE_URL. Servidor HTTP: app.listen com host 0.0.0.0 e porta por process.env.PORT. Infra alvo: API no Render e banco PostgreSQL no Supabase.
Variaveis de Ambiente Criticas	Backend (runtime): DATABASE_URL, JWT_SECRET, PORT. Backend (migrations): DIRECT_URL. Mobile: EXPO_PUBLIC_API_URL. Testes backend: TEST_DATABASE_URL (via scripts/run-prisma-test-command.ts).
Runbook de Deploy (Render)	1) Build: npm run build. 2) Migrations: npx prisma migrate deploy (com DIRECT_URL configurada). 3) Start: npm run start. 4) Validacao: checar endpoint de health funcional (pendente formalizacao no projeto).
Automacao de Entrega	Nao ha workflow de CI/CD versionado em .github/workflows no estado atual. Nao ha arquivo de infraestrutura declarativa de deploy (ex.: render.yaml) no repositório. Publicacao atual ocorre por configuracao manual do ambiente Render/Supabase.
Arquitetura Mobile (estado atual)	Navegacao por Stack em expo-router (app/_layout.tsx) com rotas para autenticacao, feed, busca, clubes, perfil, comentarios e configuracoes. Tema global por ThemeContext (system/light/dark). Camada de dados centralizada em mobile/services/api.ts com parse padrao de resposta e token em AsyncStorage.
Banco de Dados	PostgreSQL com Prisma ORM (Supabase em producao)
Controle de Versao	Git/GitHub
Servicos de Terceiros	Render (hospedagem da API backend) e Supabase (PostgreSQL gerenciado em producao).
Testes Automatizados (estado documentado)	Backend: 17 arquivos de teste em backend/tests. Mobile: 2 arquivos em mobile/__tests__. Mobile cobre login e cadastro (estado inicial, habilitacao de botao,

	chamada de API e tratamento de erro). Relatorios em docs/backend/tests e docs/mobile/tests com suites recentes em status PASS.
Escopo Mobile Mapeado	17 telas em app/, 84 componentes reutilizaveis, 10 hooks de estado/fluxo, 6 arquivos de constantes de tema e UI, 6 arquivos de tipos.

5. IDEIAS FUTURAS (BACKLOG)

Funcionalidades planejadas para versoes posteriores.

- Implementacao completa do fluxo de recuperacao de senha no backend e no app.
- Implementacao das funcionalidades completas de clubes (CRUD, membros, convites, entradas, auditoria e feeds).
- Implementacao dos prompts de clubes (publicacao, edicao, moderacao, respostas, comentarios e likes).
- Implementacao da busca com resultados reais, sugestoes e historico de buscas recentes.
- Implementacao do centro de notificacoes (listagem, leitura, badge e navegacao para conteudo relacionado).
- Implementacao das funcionalidades de configuracoes (privacidade, alterar email/senha, tema, suporte e exclusao de conta).
- Implementacao do fluxo de provas com upload assinado e visualizacao detalhada no app.

ANEXO A - EVIDENCIAS TECNICAS IMPLEMENTADAS

Backend - rotas principais

Base	Endpoints principais
/auth	POST /signup, POST /login
/users	GET /me, PUT /me, GET /
/truths	POST /, DELETE /:id
/dares	POST /, DELETE /:id
/feed	GET /
likes	POST /truths/:id/like, POST /dares/:id/like, POST /clubs/:id/like

Backend - evidencias de deploy em producao

Arquivo	Configuracao observada	Finalidade
backend/package.json	"build": "prisma generate && tsc -p tsconfig.build.json"	Gera cliente Prisma e compila artefato de producao.
backend/package.json	"start": "node dist/src/server.js" e "main": "dist/src/server.js"	Define entriypoint compilado correto para execucao em producao.
backend/tsconfig.build.json	Build dedicado incluindo src/**/*.ts e excluindo testes	Evita artefatos de teste no bundle de deploy.

Arquivo	Configuracao observada	Finalidade
backend/prisma.config.ts	datasource.url: env('DIRECT_URL')	Usa conexao direta para migrations no banco Supabase.
backend/src/lib/prisma.ts	PrismaPg com connectionString em process.env.DATABASE_URL	Conexao de runtime da API no Render ao PostgreSQL do Supabase.
backend/src/server.ts	app.listen(PORT, '0.0.0.0')	Permite binding externo exigido pelo ambiente de deploy do Render.
Ambiente de producao	Render + Supabase	Define arquitetura de nuvem atual do backend online.

Mobile - consumo atual de API e estado de integracao

Fluxo mobile	Funcoes/Endpoints consumidos	Status
Autenticacao	signup(), login(), saveToken(), getToken()	Integrado com backend
Feed	getFeed(), toggleLike(), deleteTruth(), deleteDare()	Integrado com backend (com fallback local em falhas)
Criar desafio	getUsers(), createTruth(), createDare()	Integrado com backend
Perfil	getMyProfile(), updateMyProfile()	Integrado com backend
Configuracoes	removeToken()	Integrado para logout
Busca, clubes, comentarios, criar grupo	Hooks locais (useSearchScreen, useClubsScreen, useFeedCommentsScreen, useCreateGroupScreen)	UI pronta, integracao backend pendente